Evilboy's XCPC Algorithm Template

Evilboy's XCPC Algorithm Template

- 动态规划
  - 斜率优化DP
- 图论
  - 网络流
  - 带花树
  - 缩点
- 数据结构
  - BigInt
  - 跳表
  - 吉司机线段树
  - 笛卡尔树
- 数学
  - exgcd
  - Miller-Rabin 大素数判定
  - Pollard rho 大整数分解
  - O(1) GCD
  - FWT
  - NTT
  - Min25筛
- 字符串
  - Manacher
  - ACAM
  - PAM
  - SAM

动态规划

斜率优化DP

```cpp
/*
2024_SCAU暑期个人排位赛1
evilboy_   =====   2024/7/9
Problem D
*/

#include <bits/stdc++.h>

#define x first
#define y second
#define endl '\n'
#define int long long

using namespace std;

typedef long long LL;
typedef pair<int,int> PII;


const int N = 100010, M = N * 2;
const int INF = 0x3f3f3f3f;

int n, m;

void solve() {
    cin >> n;
    vector<int> x(n + 1), p(n + 1), c(n + 1);
    vector<int> dp(n + 1);
    for(int i = 1; i <= n; i++)
        cin >> x[i] >> p[i] >> c[i];
    vector<int> cnt(n + 1), sum(n + 1);
    for(int i = 1; i <= n; i++) {
        cnt[i] = cnt[i - 1] + p[i];
        sum[i] = sum[i - 1] + p[i] * x[i];
    }

    // for(int i = 1; i <= n; i++) {
    //     dp[i] = INF;
    //     for(int j = 0; j < i; j++)
    //         dp[i] = min(dp[i], dp[j] + (cnt[i] - cnt[j]) * x[i] - (sum[i] - sum[j]) + c[i]);
    // }
```

```
    // cout << dp[n] << endl;

    /* 斜率优化DP
    dp[i] = min{ dp[j] + (cnt[i] - cnt[j]) * x[i] - (sum[i] - sum[j]) + c[i] }
          = cnt[i] * x[i] - sum[i] + c[i] + min{ dp[j] - cnt[j] * x[i] + sum[j] }
          = cnt[i] * x[i] - sum[i] + c[i] + min{ dp[j] + sum[j] - cnt[j] * x[i] }
    dp[j] + sum[j] - cnt[j] * x[i]
    令 Y = dp[j] + sum[j], K = x[i], X = cnt[j]
    则 min{ Y - KX }
    令 B = Y - KX, 即 min{ B }

    斜率DP模板
    for(int i = 1; i <= n; i++) {
        while(hh < tt && (Y[q[hh + 1]] - Y[q[hh]]) <= (X[q[hh + 1]] - X[q[hh]]) * K[i])
            hh ++;
        dp[i] = ...
        while(hh < tt && (Y[q[tt]] - Y[q[tt - 1]]) * (X[i] - X[q[tt]]) >= (Y[i] - Y[q[tt]]) * (X[q[tt]] - X[q[tt - 1]]))
            tt --;
        q[++ tt] = i;
    }
    */
    vector<int> q(n + 1);
    int hh = 0, tt = 0;
    for(int i = 1; i <= n; i++) {
        while(hh < tt && ((dp[q[hh + 1]] + sum[q[hh + 1]]) - (dp[q[hh]] + sum[q[hh]])) <= (cnt[q[hh + 1]] - cnt[q[hh]]) * x[i])
            hh ++;
        dp[i] = dp[q[hh]] + (cnt[i] - cnt[q[hh]]) * x[i] - (sum[i] - sum[q[hh]]) + c[i];
        while(hh < tt && ((dp[q[tt]] + sum[q[tt]]) - (dp[q[tt - 1]] + sum[q[tt - 1]])) * (cnt[i] - cnt[q[tt]]) >= ((dp[i] + sum[i]) - (dp[q[tt]] + sum[q[tt]])) * (cnt[q[tt]] - cnt[q[tt - 1]]))
            tt --;
        q[++ tt] = i;
    }

    int ans = dp[n];
    while(n >= 0 && p[n] == 0)
        ans = min(ans, dp[-- n]);
    cout << ans << endl;
}

signed main() {
```

```
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);

    int T = 1;
    // cin >> T;
    while(T --)
        solve();

    return 0;
}

/*
题目描述
L 公司有 n 个工厂，由高到低分布在一座山上，工厂1在山顶，工厂n在山脚。
由于这座山处于高原内陆地区(干燥少雨)，L 公司一般把产品直接堆放在露天，以节省费用。 突然有一天，
L 公司的总裁 L 先生接到气象部门的电话，被告知三天之后将有一场暴雨，于是 L 先生决定紧急在某些工
厂建立一些仓库以免产品被淋坏。
由于地形的不同，在不同工厂建立仓库的费用可能是不同的。 第 i 个工厂目前已有成品 pi 件，在第 i 个
工厂位置建立仓库的费用是 ci 。
对于没有建立仓库的工厂，其产品应被运往其他的仓库进行储藏，而由于 L 公司产品的对外销售处设置在
山脚的工厂n，故产品只能往山下运(即只能运往编号更大的工厂的仓库)，当然运送产品也是需要费用的，
一件产品运送一个单位距离的费用是 1 。
假设建立的仓库容量都是足够大的，可以容下所有的产品。 你将得到以下数据：
+ 工厂 i 距离工厂 1 的距离 xi (其中 x1 = 0)
+ 工厂 i 目前已有成品数量 pi
+ 在工厂 i 建立仓库的费用 ci
请你帮助 L 公司寻找一个仓库建设的方案，使得总的费用(建造费用 + 运输费用)最小

输入格式
输入的第一行是一个整数 n ，代表工厂的个数。
第 2 到 (n + 1) 行，每行有三个用空格隔开的整数，第 (i + 1) 行的整数依次代表 xi, pi, ci

输出格式
仅输出一行一个整数，代表最优方案的费用。

样例
3
0 5 10
5 3 100
9 6 10

32
```

```
*/
```

图论

网络流

```
// Dinic 最大流
template<typename T> struct Flow_ {
    const int n;
    const T inf = numeric_limits<T>::max();
    struct Edge {
        int to;
        T w;
        Edge(int to, T w) : to(to), w(w) {}
    };
    vector<Edge> ver;
    vector<vector<int>> h;
    vector<int> cur, d;

    Flow_(int n) : n(n), h(n + 1) {}
    void add(int u, int v, T c) {
        h[u].push_back(ver.size());
        ver.emplace_back(v, c);
        h[v].push_back(ver.size());
        ver.emplace_back(u, 0);
    }
    bool bfs(int s, int t) {
        d.assign(n + 1, -1);
        d[s] = 0;
        queue<int> q;
        q.push(s);
        while(!q.empty()) {
            auto x = q.front();
            q.pop();
            for(auto it: h[x]) {
                auto [y, w] = ver[it];
                if(w && d[y] == -1) {
                    d[y] = d[x] + 1;
                    if(y == t) return true;
                    q.push(y);
                }
            }
        }
        return false;
    }
    T dfs(int u, int t, T f) {
        if(u == t) return f;
```

```
        auto r = f;
        for(int &i = cur[u]; i < h[u].size(); i++) {
            auto j = h[u][i];
            auto &[v, c] = ver[j];
            auto &[u, rc] = ver[j ^ 1];
            if(c && d[v] == d[u] + 1) {
                auto a = dfs(v, t, std::min(r, c));
                c -= a;
                rc += a;
                r -= a;
                if(!r) return f;
            }
        }
        return f - r;
    }
    T work(int s, int t) {
        T ans = 0;
        while(bfs(s, t)) {
            cur.assign(n + 1, 0);
            ans += dfs(s, t, inf);
        }
        return ans;
    }
};
using Flow = Flow_<int>;
```

```
// EK 费用流
struct MinCostFlow {
    using LL = long long;
    using PII = pair<LL,int>;
    const LL INF = numeric_limits<LL>::max();
    struct Edge {
        int v, c, f;
        Edge(int v, int c, int f) : v(v), c(c), f(f) {}
    };
    const int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<LL> h, dis;
    vector<int> pre;

    MinCostFlow(int n) : n(n), g(n + 1) {}
    void add(int u, int v, int c, int f) { // c 流量, f 费用
        g[u].push_back(e.size());
        e.emplace_back(v, c, f);
        g[v].push_back(e.size());
        e.emplace_back(u, 0, -f);
```

```
    }
    bool dijkstra(int s, int t) {
        dis.assign(n + 1, INF);
        pre.assign(n + 1, -1);
        priority_queue<PII, vector<PII>, greater<PII>> que;
        dis[s] = 0;
        que.emplace(0, s);
        while(!que.empty()) {
            auto [d, u] = que.top();
            que.pop();
            if(dis[u] < d) continue;
            for(int i: g[u]) {
                auto [v, c, f] = e[i];
                if(c > 0 && dis[v] > d + h[u] - h[v] + f) {
                    dis[v] = d + h[u] - h[v] + f;
                    pre[v] = i;
                    que.emplace(dis[v], v);
                }
            }
        }
        return dis[t] != INF;
    }
    pair<int,LL> flow(int s, int t) {
        int flow = 0;
        LL cost = 0;
        h.assign(n + 1, 0);
        while(dijkstra(s, t)) {
            for(int i = 1; i <= n; i++)
                h[i] += dis[i];
            int aug = numeric_limits<int>::max();
            for(int i = t; i != s; i = e[pre[i] ^ 1].v)
                aug = min(aug, e[pre[i]].c);
            for(int i = t; i != s; i = e[pre[i] ^ 1].v) {
                e[pre[i]].c -= aug;
                e[pre[i] ^ 1].c += aug;
            }
            flow += aug;
            cost += LL(aug) * h[t];
        }
        return {flow, cost};
    }
};
```

带花树

```
// match[i]: i匹配的谁
// last[i]: i通过非匹配边连的上一个点是谁
```

```
int lca(int x, int y) { // 求环顶
    tmp ++; // 为了不清vis数组,每次进入函数时给vis赋的值都不一样就可以了
    while(1) {
        if(x != 0) {
            x = findfa(x);  // 先去x的环顶(处理非简单环的情况)
            if(vis[x] == tmp) return x; // x走过,已经是环顶了,返回
            vis[x] = tmp; // 标记走过
            if(match[x]) x = last[match[x]]; // 往上跳一条匹配边和非匹配边(BFS一层就是走了一条匹配边和一条非匹配边)
            else x = 0;
        }
        swap(x, y); // 交换xy
    }
}
```

```
void flower(int a, int r) { // 缩环(的一半)
    while(a != r) {
        int b = match[a], c = last[b]; // a是黑点所以先跳匹配边再跳非匹配边
        if(findfa(c) != r) last[c] = b; // 因为奇环里到底怎么匹配还不知道,干脆把原来的匹配边也记录一组非匹配边(任意相邻两个点都有可能是最终的匹配)
        if(col[b] == 2) q.push(b), col[b] = 1;
        if(col[c] == 2) q.push(c), col[c] = 1; // 环上的白点要变成黑点
        fa[findfa(a)] = findfa(b);
        fa[findfa(b)] = findfa(c); // 并查集维护父亲
        a = c; // 往上跳
    }
}
```

```
int work(int s) {
    for(int i = 1; i <= n; i++)
        last[i] = col[i] = vis[i] = 0, fa[i] = i; // 清数组
    while(!q.empty()) q.pop();
    col[s] = 1; // 给起点标成黑色
    q.push(s);
    while(!q.empty()) { // 广搜
        int x = q.front();
        q.pop();
        for(int i = head[x]; i > 0; i = edge[i].next) {
            int y = edge[i].t;
            if(match[x] == y) continue; // 走的匹配边(走回去了)
            if(findfa(x) == findfa(y)) continue;
```

```
            // 两个点在同一个已经缩过的奇环里
            if(col[y] == 2) continue; // 偶环
            if(col[y] == 1) { // 黑点——奇环
                int r = lca(x, y); //r是环顶
                if(findfa(x) != r) last[x] = y;
                if(findfa(y) != r) last[y] = x; // xy都是靠非匹配边接在一起
                flower(x, r);
                flower(y, r); // 缩花,每次缩掉花的一半
            } else if(!match[y]) { // else: col等于0——在这次增广中y没访问过;match为0,这个点是个未匹配的点——增广路已经求到了
                last[y] = x; // y通过非匹配边和x连到一起
                for(int u = y; u != 0;) { // 顺着交错路走回去
                    int v = last[u]; // last是他通过非匹配边连的上一个点
                    int w = match[v]; // 去v原来匹配的点
                    match[v] = u;
                    match[u] = v; // 匹配边和非匹配边交换——uv成为新一对
                    u = w; // 再从w开始往前走直到把这个增广路修改完
                    // 这里之所以没有改last的值是因为用过了就没用了,后面会退出work函数,再次进入就清空了
                }
                return 1;
            } else { // col等于0——在本次增广中y没访问过,且y原来有匹配
                last[y] = x; //通过未匹配边的y的前一个点为x
                col[y] = 2; //y自己是白点
                col[match[y]] = 1; //y通过匹配边连的前一个点是黑点
                q.push(match[y]); //这个黑点入队
            }
        }
    }
    return 0;
}
```

缩点

```
// scc
void tarjan(int u) {
    low[u] = dfn[u] = ++timecnt;
    stk.push(u);
    in_stk[u] = true;

    for(int i = h[u]; i != -1; i = ne[i]) {
        int j = e[i];
        if(!dfn[j]) {
            tarjan(j);
            low[u] = min(low[u], low[j]);
        } else if(in_stk[j])
            low[u] = min(low[u], dfn[j]);
```

```
    }

    if(low[u] == dfn[u]) {
        scc_cnt++;
        int y;
        do {
            y = stk.top();
            stk.pop();
            in_stk[y] = false;
            id[y] = scc_cnt;
            sizes[scc_cnt]++;
        }while (y != u);
    }
}
```

```
// e-dcc
void tarjan (int u, int from) {
    dfn[u] = low[u] = ++ timestamp;
    stk.push(u);

    for(int i = h[u]; i != -1; i = ne[i]) {
        int j = e[i];
        if(!dfn[j]) {
            tarjan(j, i);
            low[u] = min(low[u], low[j]);
            if(dfn[u] < low[j])
                is_bridge[i] = is_bridge[i ^ 1] = true;
        } else if(i != (from ^ 1))
            low[u] = min(low[u], dfn[j]);
    }

    if(dfn[u] == low[u]) {
        dcc_cnt ++;
        int y;
        do {
            y = stk.top();
            stk.pop();
            id[y] = dcc_cnt;
        }while(y != u);
    }
}
```

```
// v-dcc
void tarjan (int u) {
    dfn[u] = low[u] = ++timestamp;
    stk.push(u);
```

```cpp

    if(u == root && h[u] == -1) {
        dcc_cnt ++;
        dcc[dcc_cnt].push_back(u);
        return;
    }

    int cnt = 0;
    for(int i = h[u]; i != -1; i = -1) {
        int j = e[i];
        if(!dfn[j]) {
            tarjan(j);
            low[u] = min(low[u], low[j]);

            if(dfn[u] <= low[j]) {
                cnt++;
                if(u != root || cnt > 1) cut[u] = true;

                dcc_cnt ++;
                int y;
                do  {
                    y = stk.top();
                    stk.pop();
                    dcc[dcc_cnt].push_back(y);
                }while(y != j);

                dcc[dcc_cnt].push_back(u);
            }
        } else low[u] = min(low[u], dfn[j]);
    }
}

for(root = 1; root <= n; root++)
    if(!dfn[root])
        tarjan(root);
```

数据结构

BigInt

```cpp
#ifdef UseFFT
/***** FFT 板子 *******/
namespace FFT{
// 精度保证：系数总和小于 1e15 开longduoble
// 精度够(maybe总和小于1e10)可以不开longdouble
// 先调用fft_init()
// FFT_MAXN = 2^k
```

```cpp
// fft_init() to precalc FFT_MAXN-th roots
typedef long double db;
const int FFT_MX=2097152, N =4001000;
const db pi=acosl(-1.);
struct cp{
    db a,b;
    cp operator+(const cp&y)const{return (cp){a+y.a,b+y.b};}
    cp operator-(const cp&y)const{return (cp){a-y.a,b-y.b};}
    cp operator*(const cp&y)const{return (cp){a*y.a-b*y.b,a*y.b+b*y.a};}
    cp operator!()const{return (cp){a,-b};};
}nw[FFT_MX+1];
int bitrev[FFT_MX];

void dft(cp*a,int n,int flag=1){
    int d=0;
    while((1<<d)*n!=FFT_MX) d++;
    For(i,0,n-1) if(i < (bitrev[i]>>d))
        swap(a[i], a[bitrev[i]>>d]);
    for (int l=2;l<=n;l<<=1){
        int del=FFT_MX/l*flag;
        for (int i=0;i<n;i+=l){
            cp *le=a+i, *ri=a+i+(l>>1), *w=(flag==1) ? nw : nw+FFT_MX;
            For(k,0,(l>>1)-1){
                cp ne=*ri**w;
                *ri=*le-ne, *le=*le+ne;
                le++, ri++, w+=del;
            }
        }
    }
    if(flag!=1) For(i,0,n-1)
        a[i].a/=n, a[i].b/=n;
}
void fft_init(){
    int L=0;
    while((1<<L)!=FFT_MX) L++;
    bitrev[0]=0;
    For(i,1,FFT_MX-1)
        bitrev[i] = bitrev[i>>1]>>1 | ((i&1)<<(L-1));
    nw[0]=nw[FFT_MX]=(cp){1,0};
    For(i,0,FFT_MX)nw[i] = (cp){cosl(2*pi/FFT_MX*i), sinl(2*pi/FFT_MX*i)};
//very slow
}
// n, m 分别为a, b的最高次幂, 数组a的范围为[0, n], b为[0, m], c转整数四舍五入
void polymul(db *a, int n, db *b, int m, db *c) {
    static cp f[FFT_MX>>1], g[FFT_MX>>1], t[FFT_MX>>1];
    int N=2;
    while(N<=n+m)N<<=1;
```

```
    For(i,0,N-1)     // 此N非全局的N
        if(i&1){
            f[i>>1].b=(i<=n)?a[i]:0.0;
            g[i>>1].b=(i<=m)?b[i]:0.0;
        }else{
            f[i>>1].a=(i<=n)?a[i]:0.0;
            g[i>>1].a=(i<=m)?b[i]:0.0;
        }
    dft(f,N>>1); dft(g,N>>1);
    int del=FFT_MX/(N>>1);
    cp qua=(cp){0,0.25}, one=(cp){1,0}, four=(cp){4,0}, *w=nw;
    For(i,0,(N>>1)-1){
        int j=i?(N>>1)-i:0;
        t[i] = (four*!(f[j]*g[j])-(!f[j]-f[i])*(!g[j]-g[i])*(one+*w))*qua;
        w+=del;
    }
    dft(t,N>>1,-1ll);
    For(i,0,n+m)c[i]=(i&1) ? t[i>>1].a : t[i>>1].b;
}
// 这实际抄的时候可以和上面结合一下 少个复制的常数
db A[N], B[N], C[N];
vector<ll> vectorMul(const vector<ll>& a, const vector<ll>& b){
    int n = a.size(), m = b.size();
    if(n==0 || m==0)return {};
    n--, m--;
    For(i,0,n)A[i] = a[i];
    For(i,0,m)B[i] = b[i];
    polymul(A, n, B, m, C);
    vector<ll> res(n+m+1);
    For(i,0,n+m) res[i] = C[i]+0.5;
    return res;
}
}
#endif
```

```
namespace BigIntSP{
// 十进制高精度板子, 将 D 个十进制位压到一起
// FFT 时最好<=5
const int D = 1;
const int B = pow(10, D);
struct BigInt
{
    // ========================== 初始化部分 ==========================
    // int, string 转 BigInt; BigInt 转 string 输出; 取绝对值, 取反; 高位推进

    int sign = 0;
    vector<ll> v;
```

```
    BigInt(ll x = 0){
        set(x);
    }
    BigInt(const string &s) {
        set(s);
    }
    void set(ll x){
        v.clear();
        sign = 0;
        if (x < 0) x *= -1, sign = 1;
        while (x) {
            v.push_back(x % B);
            x /= B;
        }
    }
    void set(const string& s){
        sign = 0;
        v.clear();
        int beg = 0;
        if (s[0] == '-')
            beg++, sign = 1;
        int add = 0, cnt = 0, base = 1;
        for (int i = s.size()-1; i >= beg; i--) {
            if (cnt == D) {
                v.push_back(add);
                cnt = add = 0;
                base = 1;
            }
            add = (s[i] - '0') * base + add;
            cnt++;
            base *= 10;
        }
        if (add) v.push_back(add);
    }
    BigInt operator-() const {
        BigInt res = *this;
        res.sign ^= 1;
        return res;
    }
    BigInt abs() const {
        BigInt res = *this;
        res.sign = 0;
        return res;
    }
    ll& operator[](const int i) { return v[i]; }
    int size() const { return v.size(); }
    void norm() {        // 向高位推一遍进位
```

```
        For(i,0,(ll)v.size()-2) {
            if (v[i] >= 0) {
                v[i + 1] += v[i] / B;
                v[i] %= B;
            } else {
                int c = (-v[i] + B - 1) / B;
                v[i] += c * B;
                v[i + 1] -= c;
            }
        }
        while (!v.empty() && v.back() >= B) {
            int c = v.back() / B;
            v.back() %= B;
            v.push_back(c);
        }
        while (!v.empty() && v.back() == 0) v.pop_back();
    }
    string to_str() const {
        string res;
        if (v.empty()) return "0";
        if (sign) res += '-';
        res += to_string(v.back());
        for (int i = (ll)v.size() - 2; i >= 0; i--) {
            string add;
            int w = v[i];
            For(_,1,D){
                add += ('0' + (w % 10));
                w /= 10;
            }
            reverse(all(add));
            res += add;
        }
        return res;
    }
    friend istream& operator>>(istream &is, BigInt &x) {
        string tmp;
        is >> tmp;
        x = BigInt(tmp);
        return is;
    }
    friend ostream& operator<<(ostream &os, BigInt x) {
        os << x.to_str();
        return os;
    }


    // ========================= O(n) 运算部分 =========================
```

```
    // 高精度加法, 高精度减法, 高精度乘低精度, 高精度除低精度

    BigInt& operator+=(const BigInt &x) {
        if (sign != x.sign) {
            *this -= (-x);
            return *this;
        }
        if ((int)v.size() < (int)x.size())
            v.resize(x.size(), 0);
        For(i,0,(ll)x.size()-1)
            v[i] += x.v[i];
        norm();
        return *this;
    }
    BigInt& operator-=(const BigInt &x) {
        if (sign != x.sign) {
            *this += (-x);
            return *this;
        }
        if (abs() < x.abs()) {
            *this = x - (*this);
            sign ^= 1;
            return *this;
        }
        For(i,0,(ll)x.size()-1)
            v[i] -= x.v[i];
        norm();
        return *this;
    }
    BigInt operator*(ll x) const {  // 注意爆ll (D<=9)
        BigInt res(*this);
        if (x < 0) res.sign ^= 1, x *= -1;
        for (int i = (ll)res.v.size()-1; i >= 0; i--)
            res.v[i] *= x;
        res.norm();
        return res;
    }
    BigInt& operator/=(ll x) {
        if (x < 0) sign ^= 1, x *= -1;
        for (int i = (ll)v.size()-1; i >= 0; i--){
            if (v[i] % x != 0 && i != 0) {
                v[i - 1] += B * (v[i] % x);
            }
            v[i] /= x;
        }
        norm();
        return *this;
```

```
    }

    BigInt operator+(const BigInt &x) const { return BigInt(*this) += x; }
    BigInt operator-(const BigInt &x) const { return BigInt(*this) -= x; }
    BigInt operator*=(const int &x) { return (*this) = (*this)*x; }
    BigInt operator/(const int &x) const { return BigInt(*this) /= x; }


    // ========================== 比较函数部分 ========================

    bool gtZer(){return sign==0 && v.size();}   // return BitInt(x) > 0

    bool operator<(const BigInt &x) const {
        if (sign != x.sign) return sign > x.sign;
        if (v.size() != x.size()) {
            if (sign) return (int)x.size() < (int)v.size();
            else return (int)v.size() < (int)x.size();
        }
        Rep(i, (ll)v.size()-1, 0) if (v[i] != x.v[i]) {
            if (sign) return x.v[i] < v[i];
            else return v[i] < x.v[i];
        }
        return false;
    }
    bool operator>(const BigInt &x) const { return x < *this; }
    bool operator<=(const BigInt &x) const { return !(x < *this); }
    bool operator>=(const BigInt &x) const { return !(*this < x); }
    bool operator==(const BigInt &x) const { return !(*this < x) && !(x < *this); }
    bool operator!=(const BigInt &x) const { return !(*this == x); }


    // ========================== O(n^2) 运算部分 ========================
    // 高精度乘高精度, 高精度除高精度, 高精度模高精度, 高精度取模, 高精度平方

    BigInt operator * (const BigInt &x) const {
        #ifndef UseFFT
        return n2Mul(*this, x);
        #else
        return FFTMul(*this, x);
        #endif
        // return karatsubaMul(x);
    }
    BigInt& operator/=(BigInt x){
        int lstSign = sign ^ x.sign;
        sign = x.sign = 0;
        if ((*this) < x)
```

```
            return *this = BigInt();
        if (x == BigInt(1)){
            sign = lstSign;
            return *this;
        }
        int d = v.size() - x.size() + 1;
        BigInt inv(1LL * B * B / x.v.back()), pre(0), c;
        int cur = 2, bcur = 1;
        while (inv != pre || bcur < x.size()) {
            bcur = min(bcur << 1, x.size());
            c.v = vector<ll>(x.v.end()-bcur, x.v.end());
            pre = inv;
            inv *= ((BigInt(2) << (cur + bcur - 1)) - inv * c);
            cur = min(cur << 1, d);
            inv.v = vector<ll>(inv.v.end()-cur, inv.v.end());
        }
        inv.v = vector<ll>(inv.v.end() - d, inv.v.end());
        BigInt res = (*this) * inv;
        res >>= (v.size());
        BigInt tt = (*this) - res * x;
        while (x <= tt) {
            res += BigInt(1);
            tt -= x;
        }
        v = res.v;
        return *this;
    }
    BigInt& operator%=(const BigInt &x) {
        BigInt div = (*this) / x;
        (*this) -= div * x;
        return *this;
    }

    void divMod(const BigInt& x, BigInt& divRes, BigInt& modRes) const {
        divRes = (*this) / x;
        modRes = (*this) - divRes*x;
    }

    BigInt square() {
        BigInt res = *this;
        res.sign = 0;
        #ifdef UseFFT
        auto v1 = FFT::vectorMul(v, v);
        #else
        auto v1 = n2Mul(*this, *this);
        #endif
        res.v.assign(v1.size(), 0);
```

```
        For(i, 0, (ll)v1.size()-1) {
            ll val = v1[i];
            for (int j = i; val; j++) {
                if (j == (int)res.v.size())
                    res.v.push_back(0);
                res.v[j] += val % B;
                val /= B;
            }
        }
        res.norm();
        return res;
    }

    BigInt operator*=(const BigInt &x) { return (*this) = (*this)*x; }
    BigInt operator/(const BigInt &x) const { return BigInt(*this) /= x; }
    BigInt operator%(const BigInt &x) const { return BigInt(*this) %= x; }

private:

    /****************三种乘法*********************/
    static BigInt n2Mul(const BigInt& a, const BigInt& b) {
        BigInt res;
        res.v.resize(a.size()+b.size(), 0);
        res.sign = a.sign ^ b.sign;
        for(int i = 0; i < a.size(); i++)
            for(int j = 0; j < b.size(); j++)
                res[i+j] += a.v[i]*b.v[j];
        res.norm();
        return res;
    }
    // BigInt& karatsubaMul(const BigInt& x){
    //     // todo
    // }
    #ifdef UseFFT
    static BigInt FFTMul(const BigInt& a, const BigInt& b) {
        if(a.size()+b.size() < 1000)return n2Mul(a, b);

        BigInt res;
        res.sign = a.sign ^ b.sign;
        auto v1 = FFT::vectorMul(a.v, b.v);
        res.v.assign(v1.size(), 0);
        for(int i = 0; i < v1.size(); i++){
            ll val = v1[i];
            for (int j = i; val; j++) {
                if (j == (int)res.v.size())
                    res.v.push_back(0);
                res.v[j] += val % B;
```

```
                val /= B;
            }
        }
        res.norm();
        return res;
    }
    #endif

    /*** 这的左右移不是正常的左右移 做除法时用 ***/
    BigInt& operator<<=(const int& x){
        if(!v.empty()) {
            vector<ll> add(x, 0);
            v.insert(v.begin(), all(add));
        }
        return *this;
    }
    BigInt& operator>>=(const int& x){
        v = vector<ll>(v.begin()+min(x,(int)v.size()), v.end());
        return *this;
    }
    BigInt operator<<(const int& x)const{return BigInt(*this)<<=x;}
    BigInt operator>>(const int& x)const{return BigInt(*this)>>=x;}
};
typedef BigInt Bigint;
Bigint qmi(Bigint m, int k)
{
    Bigint res(1);
    while (k) {
        if (k & 1) res *= m;
        m *= m;
        k >>= 1;
    }
    return res;
}
}
using BigIntSP::BigInt;
using BigIntSP::qmi;
```

跳表

```
LG2[1] = 0;
for(int i = 2; i < N; i++) LG[i] = LG[i / 2] + 1;
```

```
int a[MAXN], ST_max[MAXN][16], ST_min[MAXN][16], l, r;
void Sparse_Table() {
    for(int i = 1; i <= n; i++)
        ST_max[i][0] = ST_min[i][0] = a[i];
```

```
    for(int j = 1; (1 << j) <= n; j++)
        for(int i = 1; i + (1 << j) - 1 <= n; i++) {
            ST_max[i][j] = max(ST_max[i][j - 1], ST_max[i + (1 << (j - 1))][j - 1]);
            ST_min[i][j] = min(ST_min[i][j - 1], ST_min[i + (1 << (j - 1))][j - 1]);
        }
}
int RMQ(int l, int r) {
    int k = LG2[r - l + 1];
    int maxnum = max(ST_max[l][k], ST_max[r - (1 << k) + 1][k]);
    int minnum = min(ST_min[l][k], ST_min[r - (1 << k) + 1][k]);
    return maxnum - minnum;
}
```

吉司机线段树

```
const int N = 500010;
const int inf = 0x3f3f3f3f3f3f3f3f;
int a[N];

struct Segment_Tree_Beats {
    struct node {
        int l, r;
        // 区间和, 最大值, 次大值, 最大值个数
        // 最小值, 次小值, 最小值个数, 区间加懒标记
        int sum, mx, mx2, cmx, mn, mn2, cmn, lazy;
    };
    vector<node> tr;
    void pushup(int u) {
        tr[u].sum = tr[u << 1].sum + tr[u << 1 | 1].sum;
        tr[u].mx = max(tr[u << 1].mx, tr[u << 1 | 1].mx);
        tr[u].mn = min(tr[u << 1].mn, tr[u << 1 | 1].mn);
        if(tr[u << 1].mx == tr[u << 1 | 1].mx) {
            tr[u].cmx = tr[u << 1].cmx + tr[u << 1 | 1].cmx;
            tr[u].mx2 = max(tr[u << 1].mx2, tr[u << 1 | 1].mx2);
        } else if(tr[u << 1].mx > tr[u << 1 | 1].mx) {
            tr[u].cmx = tr[u << 1].cmx;
            tr[u].mx2 = max(tr[u << 1].mx2, tr[u << 1 | 1].mx);
        } else {
            tr[u].cmx = tr[u << 1 | 1].cmx;
            tr[u].mx2 = max(tr[u << 1].mx, tr[u << 1 | 1].mx2);
        }
        if(tr[u << 1].mn == tr[u << 1 | 1].mn) {
            tr[u].cmn = tr[u << 1].cmn + tr[u << 1 | 1].cmn;
            tr[u].mn2 = min(tr[u << 1].mn2, tr[u << 1 | 1].mn2);
        } else if(tr[u << 1].mn < tr[u << 1 | 1].mn) {
```

```
            tr[u].cmn = tr[u << 1].cmn;
            tr[u].mn2 = min(tr[u << 1].mn2, tr[u << 1 | 1].mn);
        } else {
            tr[u].cmn = tr[u << 1 | 1].cmn;
            tr[u].mn2 = min(tr[u << 1].mn, tr[u << 1 | 1].mn2);
        }
    }
    void pushdown(int u) {
        if(tr[u].lazy) {
            tr[u << 1].sum += (tr[u << 1].r - tr[u << 1].l + 1) * tr[u].lazy;
            tr[u << 1].lazy += tr[u].lazy;
            tr[u << 1].mx += tr[u].lazy, tr[u << 1].mx2 += tr[u].lazy, tr[u << 1].mn += tr[u].lazy, tr[u << 1].mn2 += tr[u].lazy;
            tr[u << 1 | 1].sum += (tr[u << 1 | 1].r - tr[u << 1 | 1].l + 1) * tr[u].lazy;
            tr[u << 1 | 1].lazy += tr[u].lazy;
            tr[u << 1 | 1].mx += tr[u].lazy, tr[u << 1 | 1].mx2 += tr[u].lazy, tr[u << 1 | 1].mn += tr[u].lazy, tr[u << 1 | 1].mn2 += tr[u].lazy;
            tr[u].lazy = 0;
        }
        if(tr[u << 1].mx > tr[u].mx) {
            tr[u << 1].sum -= (tr[u << 1].mx - tr[u].mx) * tr[u << 1].cmx;
            if(tr[u << 1].mn2 == tr[u << 1].mx) tr[u << 1].mn2 = tr[u].mx;
            if(tr[u << 1].mn == tr[u << 1].mx) {
                tr[u << 1].mn = tr[u].mx, tr[u << 1].mn2 = inf, tr[u << 1].cmn = tr[u << 1].r - tr[u << 1].l + 1;
            }
            tr[u << 1].mx = tr[u].mx;
        }
        if(tr[u << 1 | 1].mx > tr[u].mx) {
            tr[u << 1 | 1].sum -= (tr[u << 1 | 1].mx - tr[u].mx) * tr[u << 1 | 1].cmx;
            if(tr[u << 1 | 1].mn2 == tr[u << 1 | 1].mx) tr[u << 1 | 1].mn2 = tr[u].mx;
            if(tr[u << 1 | 1].mn == tr[u << 1 | 1].mx) {
                tr[u << 1 | 1].mn = tr[u].mx, tr[u << 1 | 1].mn2 = inf, tr[u << 1 | 1].cmn = tr[u << 1 | 1].r - tr[u << 1 | 1].l + 1;
            }
            tr[u << 1 | 1].mx = tr[u].mx;
        }
        if(tr[u << 1].mn < tr[u].mn) {
            tr[u << 1].sum += (tr[u].mn - tr[u << 1].mn) * tr[u << 1].cmn;
            if(tr[u << 1].mx2 == tr[u << 1].mn) tr[u << 1].mx2 = tr[u].mn;
            if(tr[u << 1].mx == tr[u << 1].mn) {
                tr[u << 1].mx = tr[u].mn, tr[u << 1].mx2 = -inf, tr[u << 1].cmx = tr[u << 1].r - tr[u << 1].l + 1;
            }
```

```
            tr[u << 1].mn = tr[u].mn;
        }
        if(tr[u << 1 | 1].mn < tr[u].mn) {
            tr[u << 1 | 1].sum += (tr[u].mn - tr[u << 1 | 1].mn) * tr[u << 1 | 1].cmn;
            if(tr[u << 1 | 1].mx2 == tr[u << 1 | 1].mn) tr[u << 1 | 1].mx2 = tr[u].mn;
            if(tr[u << 1 | 1].mx == tr[u << 1 | 1].mn) {
                tr[u << 1 | 1].mx = tr[u].mn, tr[u << 1 | 1].mx2 = -inf, tr[u << 1 | 1].cmx = tr[u << 1 | 1].r - tr[u << 1 | 1].l + 1;
            }
            tr[u << 1 | 1].mn = tr[u].mn;
        }
    }
    void build(int u, int l, int r) {
        if(l == r) {
            tr[u] = {l, r, a[l], a[l], -inf, 1, a[l], inf, 1, 0};
            return;
        }
        tr[u] = {l, r};
        int mid = l + r >> 1;
        build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
        pushup(u);
    }
    Segment_Tree_Beats(int n) {
        tr.resize((n + 10) * 4);
        build(1, 1, n);
    }
    // ai <- min(ai, c)
    void modify_mn(int u, int l, int r, int c) {
        if(c >= tr[u].mx) return;
        if(tr[u].l >= l && tr[u].r <= r && c > tr[u].mx2) {
            tr[u].sum -= (tr[u].mx - c) * tr[u].cmx;
            if(tr[u].mn2 == tr[u].mx) tr[u].mn2 = c;
            if(tr[u].mn == tr[u].mx) {
                tr[u].mn = c, tr[u].mn2 = inf, tr[u].cmn = tr[u].r - tr[u].l + 1;
            }
            tr[u].mx = c;
            return;
        }
        pushdown(u);
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) modify_mn(u << 1, l, r, c);
        if(r > mid) modify_mn(u << 1 | 1, l, r, c);
        pushup(u);
    }
```

```
    // ai <- max(ai, c)
    void modify_mx(int u, int l, int r, int c) {
        if(c <= tr[u].mn) return;
        if(tr[u].l >= l && tr[u].r <= r && c < tr[u].mn2) {
            tr[u].sum += (c - tr[u].mn) * tr[u].cmn;
            if(tr[u].mx2 == tr[u].mn) tr[u].mx2 = c;
            if(tr[u].mx == tr[u].mn) {
                tr[u].mx = c, tr[u].mx2 = -inf, tr[u].cmx = tr[u].r - tr[u].l + 1;
            }
            tr[u].mn = c;
            return;
        }
        pushdown(u);
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) modify_mx(u << 1, l, r, c);
        if(r > mid) modify_mx(u << 1 | 1, l, r, c);
        pushup(u);
    }
    // ai <- ai + c
    void modify_add(int u, int l, int r, int c) {
        if(tr[u].l >= l && tr[u].r <= r) {
            tr[u].sum += (tr[u].r - tr[u].l + 1) * c;
            tr[u].lazy += c;
            tr[u].mx += c, tr[u].mx2 += c, tr[u].mn += c, tr[u].mn2 += c;
            return;
        }
        pushdown(u);
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) modify_add(u << 1, l, r, c);
        if(r > mid) modify_add(u << 1 | 1, l, r, c);
        pushup(u);
    }
    int query_sum(int u, int l, int r) {
        if(tr[u].l >= l && tr[u].r <= r)
            return tr[u].sum;
        pushdown(u);
        int mid = tr[u].l + tr[u].r >> 1, res = 0;
        if(l <= mid) res += query_sum(u << 1, l, r);
        if(r > mid) res += query_sum(u << 1 | 1, l, r);
        return res;
    }
    int query_mx(int u, int l, int r) {
        if(tr[u].l >= l && tr[u].r <= r)
            return tr[u].mx;
        pushdown(u);
        int mid = tr[u].l + tr[u].r >> 1, res = -inf;
```

```
        if(l <= mid) res = max(res, query_mx(u << 1, l, r));
        if(r > mid) res = max(res, query_mx(u << 1 | 1, l, r));
        return res;
    }
    int query_mn(int u, int l, int r) {
        if(tr[u].l >= l && tr[u].r <= r)
            return tr[u].mn;
        pushdown(u);
        int mid = tr[u].l + tr[u].r >> 1, res = inf;
        if(l <= mid) res = min(res, query_mn(u << 1, l, r));
        if(r > mid) res = min(res, query_mn(u << 1 | 1, l, r));
        return res;
    }
};

void solve()
{
    int n, m;
    cin >> n;
    for(int i = 1; i <= n; i++)
        cin >> a[i];

    Segment_Tree_Beats Tr(n);
    cin >> m;
    while(m --) {
        int op;
        cin >> op;
        if(op == 1) {
            int l, r, c;
            cin >> l >> r >> c;
            Tr.modify_add(1, l, r, c);
        } else if(op == 2) {
            int l, r, c;
            cin >> l >> r >> c;
            Tr.modify_mx(1, l, r, c);
        } else if(op == 3) {
            int l, r, c;
            cin >> l >> r >> c;
            Tr.modify_mn(1, l, r, c);
        } else if(op == 4) {
            int l, r;
            cin >> l >> r;
            cout << Tr.query_sum(1, l, r) << '\n';
        } else if(op == 5) {
            int l, r;
            cin >> l >> r;
            cout << Tr.query_mx(1, l, r) << '\n';
```

```
        } else {
            int l, r;
            cin >> l >> r;
            cout << Tr.query_mn(1, l, r) << '\n';
        }
    }
}
```

笛卡尔树

```
int n = 11;
int h[] = {0, 9, 3, 7, 1, 8, 12, 10, 20, 15, 18, 5};
int lson[N], rson[N], root;

void build() {
    stack<int> stk;
    for(int i = 1; i <= n; i++) {
        while(!stk.empty() && h[stk.top()] >= h[i])
            lson[i] = stk.top(), stk.pop();
        if(!stk.empty()) rson[stk.top()] = i;
        stk.push(i);
    }
    while(!stk.empty()) root = stk.top(), stk.pop();
}
```

数学

exgcd

```
ll gcd(ll a, ll b) {
    if (b == 0) return a;
    else return gcd(b, a % b);
}
ll exgcd(ll a, ll b, ll &x, ll &y) { //返回gcd(a,b)
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
}

// 解方程 ax+by=c 最小非负 x 解
int solve_equation(ll a, ll b, ll c, ll& x, ll& y) {
    // ax+by=gcd(a,b)
```

```
    int g = exgcd(a, b, x, y);
    if(c % g != 0){ // 无解
        x = y = 0;
        return 0;
    }
    int t = b / g, k = a / g, d = c / g;
    x *= d; // 将方程转为 ax+by=c
    x = (x % t + t) % t;
    y = (c - a * x) / b;
     /*
     // 最小非负 y
     y *= d;
     y = (y % k + k) % k;
     x = (c - b * y) / a;
     */
     return g;
}
```

Miller-Rabin 大素数判定

```
#include <bits/stdc++.h>
using namespace std;
using i128 = __int128;

namespace Miller_Rabin {
    inline i128 read() {
        i128 x = 0, f = 1;
        char ch = getchar();
        while(ch < '0' || ch > '9') {
            if(ch == '-') f = -1;
            ch = getchar();
        }
        while(ch >= '0' && ch <= '9') {
            x = (x << 1) + (x << 3) + (ch - '0');
            ch = getchar();
        }
        return x * f;
    }

    void print(i128 x) {
        if(x < 0) {
            putchar('-');
            x = -x;
        }
        if(x > 9) print(x / 10);
        putchar(x % 10 + '0');
    }
```

```cpp

    i128 mul(i128 a, i128 b, i128 p) {
        a %= p, b %= p;
        i128 ans = 0;
        while(b) {
            if(b & 1) ans = (ans + a) % p;
            a = (a + a) % p;
            b >>= 1;
        }
        return ans;
    }

    i128 qmi(i128 a, i128 b, i128 p) {
        i128 ans = 1;
        while(b) {
            if(b & 1) ans = mul(ans, a, p);
            a = mul(a, a, p);
            b >>= 1;
        }
        return ans;
    }

    // 2, 7, 61 适用 4e9
    // 2, 3, 5, 7, 11, 13, 17 适用 3e14
    // 2, 3, 7, 61, 24251 适合 1e16 特判 46856248255981
    static constexpr int primelist[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    bool miller_rabin(i128 n)  {
        if (n < 3 || n % 2 == 0) return n == 2;
        i128 a = n - 1, b = 0;
        while (a % 2 == 0) a >>= 1, b++;

        for (int i : primelist)  {
            i128 v = qmi(i, a, n);
            if(v == 1) continue;
            for (int j = 0; j < b || (b = -1, false); j++)  {
                if(v == n - 1) break;
                v = mul(v, v, n);
            }
            if (b < 0) return false;
        }
        return true;
    }
}
using namespace Miller_Rabin;

signed main() {
```

```
    i128 n = read();
    if(miller_rabin(n)) puts("Yes");
    else puts("No");
}
```

Pollard rho 大整数分解

```
namespace Let_it_Rot {
    typedef long long ll;
    using f64 = long double;

    ll p;
    f64 invp;

    void setmod(ll x) {
        p = x, invp = (f64) 1 / x;
    }

    ll mul(ll a, ll b) {
        ll z = a * invp * b + 0.5;
        ll res = a * b - z * p;
        return res + (res >> 63 & p);
    }

    ll pow(ll a, ll x, ll res = 1) {
        for(; x; x >>= 1, a = mul(a, a))
            if(x & 1) res = mul(res, a);
        return res;
    }

    bool checkprime(ll p) {
        if(p == 1) return 0;
        setmod(p);
        ll d = __builtin_ctzll(p - 1), s = (p - 1) >> d;
        for(ll a: {2, 3, 5, 7, 11, 13, 82, 373}) {
            if(a % p == 0) continue;
            ll x = pow(a, s), y;
            for(int i = 0; i < d; i++, x = y) {
                y = mul(x, x);
                if(y == 1 && x != 1 && x != p - 1)
                    return 0;
            }
            if(x != 1) return 0;
        }
        return 1;
    }

```

```
    ll rho(ll n) {
        if(!(n & 1)) return 2;
        static std:: mt19937_64 gen(((size_t)"evilboy"));
        ll x = 0, y = 0, prod = 1;
        auto f = [&](ll o) {return mul(o, o) + 1;};
        setmod(n);
        for(int t = 30, z = 0; t % 64 || std::__gcd(prod, n) == 1; t++) {
            if(x == y) x = ++z, y = f(x);
            if(ll q = mul(prod, x + n - y)) prod = q;
            x = f(x), y = f(f(y));
        }
        return std::__gcd(prod, n);
    }
    std::vector<ll> factor(ll x) {
        std::vector<ll> res;
        auto f = [&](auto f, ll x) {
            if(x == 1) return;
            if(checkprime(x)) return res.push_back(x);
            ll y = rho(x);
            f(f, y), f(f, x / y);
        };
        f(f, x), sort(res.begin(), res.end());
        return res;
    }
}
using namespace Let_it_Rot;
```

O(1) GCD

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

namespace O1_GCD {
    const int V = 1000010, T = 1010; // V 表示值域
    int pre[T + 5][T + 5];
    array<int,3> fac[V + 5];
    bool isnp[V + 5];
    int p[V + 5], tot = 0;
    int TT;
    // 预处理
    // fac 表示一个分解, isnp 表示是不是合数, p 是质数集, pre 是预处理的 gcd, V 和 v 都是值域
    void init(const int v = V) {
        fac[1] = {1, 1, 1};
        for(int i = 2; i <= v; i++) {
            if(!isnp[i]) fac[i] = {i, 1, 1}, p[tot ++] = i;
```

```
            for(int j = 0; j < tot && i * p[j] <= v; j++) {
                int t = i * p[j];
                isnp[t] = true;
                fac[t] = fac[i];
                *min_element(fac[t].begin(), fac[t].end()) *= p[j];
                if(i % p[j] == 0) break;
            }
        }
        TT = sqrt(v);
        for(int i = 1; i <= TT; i++)
            pre[0][i] = pre[i][0] = i;
        for(int i = 1; i <= TT; i++)
            for(int j = 1; j <= i; j++)
                pre[i][j] = pre[j][i] = pre[j][i % j];
    }
    // 查询
    int g(int x, int y) {
        int t = x % y;
        if(t == 0) return y;
        return y > TT ? 1 : pre[y][t];
    }
    int gcd(int a,int b) {
        if(a == 1 || b == 1) return 1;
        int a0 = g(a, fac[b][0]), a1 = g(a /= a0, fac[b][1]), a2 = g(a /= a1, fac[b][2]);
        return a0 * a1 * a2;
    }
}
using namespace O1_GCD;

mt19937 rnd(time(0));
void solve() {
    init();

    for(int t = 0; t < 1000; t++) {
        int a = rnd() % (V - 100) + 1, b = rnd() % (V - 100) + 1;
        cout << a << ' ' << b << ' ' << gcd(a, b) << ' ' << __gcd(a, b) << '\n';
        assert(gcd(a, b) == __gcd(a, b));
    }
}
```

FWT

```cpp
namespace FWT {
    const int mod = 998244353;
    // 与卷积变换
    void AND(vector<int>& a, int base = 1) {
        int n = a.size();
        for(int o = 2, k = 1; o <= n; o <<= 1, k <<= 1)
            for(int i = 0; i < n; i += o)
                for(int j = 0; j < k; j++)
                    a[i + j] = (a[i + j] + a[i + j + k] * base % mod) % mod;
    }
    // 或卷积
    void OR(vector<int>& a, int base = 1) {
        int n = a.size();
        for(int o = 2, k = 1; o <= n; o <<= 1, k <<= 1)
            for(int i = 0; i < n; i += o)
                for(int j = 0; j < k; j++)
                    a[i + j + k] = (a[i + j + k] + a[i + j] * base % mod) % mod;
    }
    // 异或卷积
    void XOR(vector<int>& a, int base = 1) {
        int n = a.size();
        for(int o = 2, k = 1; o <= n; o <<= 1, k <<= 1)
            for(int i = 0; i < n; i += o)
                for(int j = 0; j < k; j++) {
                    a[i + j] = (a[i + j] + a[i + j + k]) % mod;
                    a[i + j + k] = (a[i + j] - a[i + j + k] * 2ll % mod + mod) % mod;
                    a[i + j] = a[i + j] * base % mod;
                    a[i + j + k] = a[i + j + k] * base % mod;
                }
    }
    // 线性乘法
    void mul(vector<int>& a, vector<int>& b) {
        int n = a.size();
        for(int i = 0; i < n; i++)
            a[i] = a[i] * b[i] % mod;
    }
    int qmi(int m, int k) {
        int res = 1;
        while(k) {
            if(k & 1) res = res * m % mod;
            m = m * m % mod;
            k >>= 1;
        }
```

```
        return res;
    }
    void GET_AND(vector<int>& a, vector<int>& b) {
        AND(a), AND(b), mul(a, b), AND(a, mod - 1);
    }
    void GET_OR(vector<int>& a, vector<int>& b) {
        OR(a), OR(b), mul(a, b), OR(a, mod - 1);
    }
    void GET_XOR(vector<int>& a, vector<int>& b) {
        XOR(a), XOR(b), mul(a, b), XOR(a, qmi(2, mod - 2));
    }
}
```

NTT

```
#define mod 998244353
typedef long long ll;
struct mint{int _;mint(){_=0;}mint(ll a){_=a%mod;if(_<0)_+=mod;}void read()
{cin>>_;}};
mint operator+(mint a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-(mint a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*(mint a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint operator+=(mint& a,mint b){a._+=b._;if(a._>=mod)a._-=mod;return a;}
mint operator-=(mint& a,mint b){a._-=b._;if(a._<0)a._+=mod;return a;}
mint operator*=(mint& a,mint b){a._=(int)((ll)a._*b._%mod);return a;}
mint pow(mint a,int b){mint c=1;while(b>0){if(b&1)c*=a;a*=a;b>>=1;}return c;}
mint inv(mint a){return pow(a,mod-2);}
mint fact[N],infact[N];
mint C(int a,int b){if(b<0||b>a)return 0;return fact[a]*infact[b]*infact[a-b];}
mint invC(int a,int b){if(b<0||b>a)return 0;return infact[a]*fact[b]*fact[a-
b];}
void initC(int n){fact[0]=1;for(int i=1;i<=n;i++)fact[i]=fact[i-1]*i;
infact[n]=inv(fact[n]);for(int i=n;i>=1;i--)infact[i-1]=infact[i]*i;}

namespace NTT {
    mint g = 3;
    mint gi = inv(g);
    int rev[N];

    void ntt(mint A[], int tot, int type) {
        for(int i = 0; i < tot; i++)
            if(i < rev[i]) swap(A[i], A[rev[i]]);
        for(int mid = 1; mid < tot; mid <<= 1) {
            mint gn = pow(type > 0 ? g : gi, (mod - 1) / (mid << 1));
            int R = mid << 1;
            for(int i = 0; i < tot; i+=R) {
                mint g0 = 1;
```

```
                for(int j = i; j < mid + i; j++, g0*= gn) {
                    mint x = A[j], y = g0 * A[j + mid];
                    A[j] = x + y;
                    A[j + mid] = x - y;
                }
            }
        }
        if(type == -1) {
            mint invtot = inv(tot);
            for(int i = 0; i < tot; i++)
                A[i] *= invtot;
        }
    }

    void polymul_ntt(mint a[], mint b[], int n, int m) {
        int tot = 1, l = 0;
        while(tot <= n + m) {
            tot <<= 1, l ++;
        }
        for(int i = 0; i <= tot; i++)
            rev[i] = 0;
        for(int i = 0; i <= tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
        ntt(a, tot, 1);
        ntt(b, tot, 1);
        for(int i = 0; i <= tot; i++)
            a[i] *= b[i];
        ntt(a, tot, -1);
        ntt(b, tot, -1);
    }

    void polymul_ntt_self(mint a[], int n) {
        int tot = 1, l = 0;
        while(tot <= n * 2) {
            tot <<= 1, l ++;
        }
        for(int i = 0; i <= tot; i++)
            rev[i] = 0;
        for(int i = 0; i <= tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
        ntt(a, tot, 1);
        for(int i = 0; i <= tot; i++)
            a[i] *= a[i];
        ntt(a, tot, -1);
    }
}
```

Min25筛

```
namespace min25 {
    const int MAXN = 2e6 + 10;
    int Lim, vis[MAXN], prime[MAXN], tot;
    int N, g[MAXN], id[MAXN], cnt, pos1[MAXN], pos2[MAXN];

    void Get(int N) {
        vis[1] = 1;
        for(int i = 2; i <= N; i++) {
            if(!vis[i]) prime[++ tot] = i;
            for(int j = 1; j <= tot && i * prime[j] <= N; j++) {
                vis[i * prime[j]] = 1;
                if(!(i % prime[j])) break;
            }
        }
    }
    int get(int x) {
        return x <= Lim ? pos1[x] : pos2[N / x];
    }
    int work(int n) {   // 求区间 [1, n] 内的质数个数
        N = n, Lim = sqrt(N);
        cnt = tot = 0;
        Get(Lim);
        for(int i = 1, j; i <= N; i = N / j + 1) {
            j = N / i; id[++ cnt] = j; g[cnt] = id[cnt] - 1;
            j <= Lim ? pos1[j] = cnt : pos2[N / j] = cnt;
        }
        for(int j = 1; j <= tot; j++)
            for(int i = 1; prime[j] * prime[j] <= id[i]; i++)
                g[i] -= g[get(id[i] / prime[j])] - (j - 1);
        return g[1];
    }
}
```

```
const int mod = 1e9 + 7;
const int inv2 = 500000004, inv6 = 166666668;
const int N = 1000010;

int n, sqr;
int st[N], prime[N], cnt;  // 线性筛
int sp1[N], sp2[N]; // 质数 1次项, 2次项前缀和
int ind1[N], ind2[N], g1[N], g2[N], w[N], tot;

// 预处理, 线性筛
void get_primes(int n) {
    st[1] = 1;
```

```
    for(int i = 2; i <= n; i++) {
        if(!st[i]) {
            prime[++ cnt] = i;
            sp1[cnt] = (sp1[cnt - 1] + i) % mod;
            sp2[cnt] = (sp2[cnt - 1] + i * i % mod) % mod;
        }
        for(int j = 1; j <= cnt && prime[j] <= n / i; j++) {
            st[i * prime[j]] = 1;
            if(i % prime[j] == 0) break;
        }
    }
}

void init() {
    // 线性筛
    get_primes(sqr);

    // 第一部分预处理
    for(int i = 1; i <= n; ) {
        int j = n / (n / i);
        w[++ tot] = n / i;  // w 后面要做除法，不能取模
        int val = w[tot] % mod;
        g1[tot] = val * (1 + val) % mod * inv2 % mod;
        g2[tot] = val * (1 + val) % mod * (2 * val + 1) % mod * inv6 % mod;
        g1[tot] = (g1[tot] - 1 + mod) % mod;
        g2[tot] = (g2[tot] - 1 + mod) % mod;
        if(n / i <= sqr) ind1[n / i] = tot;
        else ind2[n / (n / i)] = tot;
        i = j + 1;
    }
    // g1, g2 分别表示一次项和二次项，ind1 和 ind2 用来记录这个数在数组中的位置
    // 由于 g 数组可以滚动，所以就只开一维
    for(int i = 1; i <= cnt; i++) {
        for(int j = 1; j <= tot && prime[i] <= w[j] / prime[i]; j++) {
            int k = w[j] / prime[i] <= sqr ? ind1[w[j] / prime[i]] : ind2[n / (w[j] / prime[i])];
            g1[j] -= prime[i] * (g1[k] - sp1[i - 1] + mod) % mod;
            g2[j] -= prime[i] * prime[i] % mod * (g2[k] - sp2[i - 1] + mod) % mod;
            g1[j] = (g1[j] % mod + mod) % mod;
            g2[j] = (g2[j] % mod + mod) % mod;
        }
    }
}

// 第二部分，求解答案
int S(int x, int y) {
```

```
    if(prime[y] >= x) return 0;
    int k = x <= sqr ? ind1[x] : ind2[n / x];
    int res = ((g2[k] - g1[k] - (sp2[y] - sp1[y])) % mod + mod) % mod;
    for(int i = y + 1; i <= cnt && prime[i] <= x / prime[i]; i++) {
        int p = prime[i];
        for(int e = 1; p <= x; e++, p *= prime[i]) {
            int val = p % mod;
            res = (res + val * (val - 1) % mod * (S(x / p, i) + (e != 1)) % mod) % mod;
        }
    }
    return res;
}

void solve() {
    cin >> n;
    sqr = sqrt(n);
    init();
    cout << (S(n, 0) + 1) % mod << '\n';
}
```

字符串

Manacher

```
struct Manacher{
    int n, N;
    string s;
    vector<int> lc;
    vector<char> ch;
    Manacher(string _s){init(_s);manacher();}
    /* s 1 bas */ /* lc 0 bas */
    void init(string _s){
        n = _s.size();
        s = " " + _s;
        lc.resize(n * 2 + 10), ch.resize(n * 2 + 10);
        ch[n * 2 + 1] = '#';
        ch[0] = '@';
        ch[n * 2 + 2] = '\0';
        for(int i = n; i >= 1; i--) {
            ch[i * 2] = s[i];
            ch[i * 2 - 1] = '#';
        }
        N = 2 * n +1;
    }
    void manacher(){
        lc[1] = 1;
```

```
        int k = 1;
        for(int i = 2; i <= N; i++) {
            int p = k + lc[k] - 1;
            if(i <= p) {
                lc[i] = min(lc[2 * k - i], p - i + 1);
            } else {
                lc[i] = 1;
            }
            while(ch[i + lc[i]] == ch[i - lc[i]]) lc[i] ++;
            if(i + lc[i] > k + lc[k]) k = i;
        }
    }
    void debug(){
        for(int i = 0; ch[i] != '\0'; i++)
            cout << ch[i];
        cout << '\n';
        for (int i = 1; i <= N; i++) {
            cout << "lc[" << i << "] = " << lc[i] << '\n';
        }
    }
};
```

ACAM

```
struct AhoCorasick {
    // SIZE: 字符集大小
    // OFFSET: 字符偏移量
    static constexpr int SIZE = 26;
    static constexpr char OFFSET = 'a';
    struct Node {
        // link: 失败链接, 用于在匹配失败时快速跳转至最长后缀匹配的状态
        // len: 从根节点到当前节点的路径长度 (即匹配的字符数)
        // next: 存储子节点索引 (当前节点通过某个字符可达到的下一个节点)
        int link, len;
        std::array<int, SIZE> next;
        Node() : link{0}, len{0}, next{} {}
    };
    std::vector<Node> t;
    AhoCorasick() { init(); }
    void init() { t.assign(1, {}); }
    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }
    int insert(const std::vector<int> &a) {
        int p = 0;
        for (auto a : a) {
```

```
            if (next(p, a) == 0) {
                t[p].next[a] = newNode();
                t[next(p, a)].len = len(p) + 1;
            }
            p = next(p, a);
        }
        return p;
    }
    int insert(const std::string &s) {
        std::vector<int> a(s.size());
        for (int i = 0; i < s.size(); i++) {
            a[i] = s[i] - OFFSET;
        }
        return insert(a);
    }
    void build() {
        std::queue<int> que;
        for (int i = 0; i < SIZE; i++) {
            if (next(0, i) != 0) {
                que.push(next(0, i));
            }
        }
        while (!que.empty()) {
            int u = que.front();
            que.pop();
            for (int i = 0; i < SIZE; i++) {
                if (next(u, i) == 0) {
                    t[u].next[i] = next(link(u), i);
                } else {
                    t[next(u, i)].link = next(link(u), i);
                    que.push(next(u, i));
                }
            }
        }
    }
    int link(int p) { return t[p].link; }
    int len(int p) { return t[p].len; }
    int next(int p, int a) { return t[p].next[a]; }
    int next(int p, char c) { return next(p, c - OFFSET); }
    int size() { return t.size(); }
} ac;
```

PAM

```
struct PAM {
    // SIZE: 字符集大小
    // OFFSET: 字符偏移量
    static constexpr int SIZE = 26;
    static constexpr char OFFSET = 'a';
    struct Node {
        // len: 当前节点代表的回文子串的长度
        // link: 后缀链接(指向除自身的最长回文后缀对应节点)
        // cnt: 当前节点为结尾的本质不同回文子串的数量(累计计数)
        // tot: 当前回文子串的出现次数
        // next: 从当前节点出发, 两端添加某字符后形成的新回文子串对应的节点索引
        int len, link, cnt, tot;
        array<int, SIZE> next;
        Node() : len{0}, link{0}, cnt{0}, tot{0}, next{} {}
    };
    // suf: 指向当前字符串中以最后一个字符结尾的最长回文子串对应的节点
    // s: 存储正在处理的字符串
    // t: 存储所有节点
    int suf;
    string s;
    vector<Node> t;
    PAM() { init(); }
    void init() {
        suf = 1;
        s.clear();
        t.assign(2, {});
        t[0].len = -1;
    }
    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }
    bool add(int x) {
        int pos = s.size();
        s.push_back(OFFSET + x);
        int cur = suf, len = 0;
        while (true) {
            len = t[cur].len;
            if (pos - len - 1 >= 0 && s[pos - len - 1] == s[pos]) {
                break;
            }
            cur = t[cur].link;
        }
        if (t[cur].next[x]) {
            suf = t[cur].next[x];
```

```
            t[suf].tot++;
            return false;
        }
        int num = newNode();
        suf = num;
        t[num].len = t[cur].len + 2;
        t[cur].next[x] = num;
        if (t[num].len == 1) {
            t[num].link = 1;
            t[num].cnt = 1;
            t[num].tot = 1;
            return true;
        }
        while (true) {
            cur = t[cur].link;
            len = t[cur].len;
            if (pos - len - 1 >= 0 && s[pos - len - 1] == s[pos]) {
                t[num].link = t[cur].next[x];
                break;
            }
        }
        t[num].cnt = t[t[num].link].cnt + 1;
        t[num].tot = 1;
        return true;
    }
    bool add(char c) { return add(c - OFFSET); }
    int len(int p) { return t[p].len; }
    int link(int p) { return t[p].link; }
    int cnt(int p) { return t[p].cnt; }
    int tot(int p) { return t[p].tot; }
    int next(int p, int x) { return t[p].next[x]; }
    int next(int p, char c) { return next(p, c - OFFSET); }
    int size() { return t.size(); }
} pam;
```

SAM

```
struct SAM {
    // SIZE: 字符集大小
    // OFFSET: 字符偏移量
    static constexpr int ALPHABET_SIZE = 26;
    static constexpr char OFFSET = 'a';
    struct Node {
        // len: 当前状态能表示的最长子串长度
        // link: 后缀链接, 表示当前状态的"最短后缀"对应的状态
        // next: 转移数组
        // cnt: 当前状态所代表的子串集合在原字符串中出现的次数
```

```
        int len;
        int link;
        int cnt;
        std::array<int, ALPHABET_SIZE> next;
        Node() : len{}, link{}, cnt{}, next{} {}
    };
    std::vector<Node> t;
    vector<vector<int>> G;
    SAM() { init(); }
    void init() {
        t.assign(2, Node());
        t[0].next.fill(1);
        t[0].len = -1;
    }
    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }
    int extend(int p, int c) {
        if (t[p].next[c]) {
            int q = t[p].next[c];
            if (t[q].len == t[p].len + 1) {
                return q;
            }
            int r = newNode();
            t[r].len = t[p].len + 1;
            t[r].link = t[q].link;
            t[r].next = t[q].next;
            t[q].link = r;
            while (t[p].next[c] == q) {
                t[p].next[c] = r;
                p = t[p].link;
            }
            return r;
        }
        int cur = newNode();
        t[cur].cnt = 1;
        t[cur].len = t[p].len + 1;
        while (!t[p].next[c]) {
            t[p].next[c] = cur;
            p = t[p].link;
        }
        t[cur].link = extend(p, c);
        return cur;
    }
    int extend(int p, char c) { return extend(p, c - OFFSET); }
    int next(int p, int x) { return t[p].next[x]; }
```

```
    int next(int p, char c) { return next(p, c - OFFSET); }
    int link(int p) { return t[p].link; }
    int len(int p) { return t[p].len; }
    int cnt(int p) { return t[p].cnt; }
    int size() { return t.size(); }

    void buildGraph() {
        G.resize(size());
        for(int i = 1; i < size(); i++)
            G[t[i].link].push_back(i);
    }
    void dfs(int p = 1) {
        for(auto i: G[p])
            dfs(i), t[p].cnt += t[i].cnt;
    }
} sam;
```